```
from google.colab import drive

drive.mount('/content/drive')

import os

print(os.listdir("/content/drive/MyDrive"))

drive_path = "/content/drive/MyDrive/Tea_Leaf_Dataset"

print(os.listdir(drive_path))

zip_path = "/content/drive/MyDrive/TLDB.zip"

import zipfile
import os

zip_path = "/content/drive/MyDrive/Tea_Leaf_Dataset/TLDB.zip"
extract_path = "/content/TLDB"

os.makedirs(extract_path, exist_ok=True)

with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall(extract_path)

print("Extraction completed!")

import os

for root, dirs, files in os.walk("/content/TLDB"):
    print(root, "->", len(files), "files")

for item in os.listdir("/content/TLDB"):
    full_path = os.path.join("/content/TLDB", item)

    if os.path.isdir(full_path):
        print("FOLDER:", item)
    else:
        print("FILE:", item)

import os

path = "/content/TLDB/TeaLeafDiseaseBD Annotated smartphone images and
l"

for item in os.listdir(path):
    full_path = os.path.join(path, item)

    if os.path.isdir(full_path):
        print("FOLDER:", item)
    else:
        print("FILE:", item)
```

```
for root, dirs, files in os.walk(path):
    image_count = sum(
        1 for f in files
        if f.lower().endswith((".jpg", ".jpeg", ".png"))
    )

    if image_count > 0:
        print(root, "→", image_count, "images")

import zipfile
import os

inner_zip = "/content/TLDB/TeaLeafDiseaseBD Annotated smartphone
images and l/TeaLeafDiseaseBD.zip"

actual_dataset = "/content/TeaLeafDataset"

os.makedirs(actual_dataset, exist_ok=True)

with zipfile.ZipFile(inner_zip, "r") as zip_ref:
    zip_ref.extractall(actual_dataset)

print("Actual dataset extracted!")

dataset_path =
"/content/TeaLeafDataset/TeaLeafDiseaseBD/TeaLeafDiseaseBD/root/TeaLea
fDataset/Original images"

print(dataset_path)

import os

print(os.listdir(dataset_path))

from PIL import Image
import os

for class_name in os.listdir(dataset_path):

    class_path = os.path.join(dataset_path, class_name)

    if os.path.isdir(class_path):

        images = [
            f for f in os.listdir(class_path)
            if f.lower().endswith((".jpg", ".jpeg", ".png"))
        ]

        print(class_name, ":", len(images))

import os
import shutil
```

```
import random

output_dir = "/content/tea_vit"

random.seed(42)

for split in ["train", "val", "test"]:
    os.makedirs(
        os.path.join(output_dir, split),
        exist_ok=True
    )

for class_name in os.listdir(dataset_path):

    class_path = os.path.join(dataset_path, class_name)

    if not os.path.isdir(class_path):
        continue

    images = [
        f for f in os.listdir(class_path)
        if f.lower().endswith((".jpg", ".jpeg", ".png"))
    ]

    random.shuffle(images)

    n = len(images)

    train_end = int(0.70 * n)
    val_end = int(0.85 * n)

    train_images = images[:train_end]
    val_images = images[train_end:val_end]
    test_images = images[val_end:]

    for split, split_images in [
        ("train", train_images),
        ("val", val_images),
        ("test", test_images)
    ]:

        destination = os.path.join(
            output_dir,
            split,
            class_name
        )

        os.makedirs(destination, exist_ok=True)

        for image in split_images:
```

```
            shutil.copy(
                os.path.join(class_path, image),
                os.path.join(destination, image)
            )

print("Dataset split completed!")

for split in ["train", "val", "test"]:

    print("\n", split.upper())

    split_path = os.path.join(output_dir, split)

    total = 0

    for class_name in os.listdir(split_path):

        class_path = os.path.join(split_path, class_name)

        count = len([
            f for f in os.listdir(class_path)
            if f.lower().endswith((".jpg", ".jpeg", ".png"))
        ])

        total += count

        print(class_name, ":", count)

    print("Total:", total)

!pip install -q timm

import torch
import torchvision
import timm
import os

print("PyTorch:", torch.__version__)
print("Torchvision:", torchvision.__version__)
print("timm:", timm.__version__)

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Device:", device)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

from torchvision import transforms
```

```
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),

    transforms.RandomHorizontalFlip(p=0.5),

    transforms.RandomRotation(15),

    transforms.ColorJitter(
        brightness=0.2,
        contrast=0.2,
        saturation=0.2
    ),

    transforms.ToTensor(),

    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

from torchvision import transforms

train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(15),
    transforms.ColorJitter(
        brightness=0.2,
        contrast=0.2,
        saturation=0.2
    ),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

val_test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

print("Transforms created successfully!")
```

```python
from torchvision import datasets
import os

output_dir = "/content/tea_vit"

train_dataset = datasets.ImageFolder(
    os.path.join(output_dir, "train"),
    transform=train_transform
)

val_dataset = datasets.ImageFolder(
    os.path.join(output_dir, "val"),
    transform=val_test_transform
)

test_dataset = datasets.ImageFolder(
    os.path.join(output_dir, "test"),
    transform=val_test_transform
)

print("Train:", len(train_dataset))
print("Validation:", len(val_dataset))
print("Test:", len(test_dataset))

print("\nClasses:")
print(train_dataset.classes)

from torch.utils.data import DataLoader

batch_size = 32

train_loader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True,
    num_workers=2,
    pin_memory=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=batch_size,
    shuffle=False,
    num_workers=2,
    pin_memory=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=batch_size,
```

```
    shuffle=False,
    num_workers=2,
    pin_memory=True
)

print("DataLoaders created!")

images, labels = next(iter(train_loader))

print("Images shape:", images.shape)
print("Labels shape:", labels.shape)
print("Labels:", labels[:10])

import matplotlib.pyplot as plt
import torch

images, labels = next(iter(train_loader))

plt.figure(figsize=(12, 8))

for i in range(8):

    plt.subplot(2, 4, i + 1)

    image = images[i].permute(1, 2, 0)

    # Undo ImageNet normalization
    image = image * torch.tensor(
        [0.229, 0.224, 0.225]
    ) + torch.tensor(
        [0.485, 0.456, 0.406]
    )

    image = torch.clamp(image, 0, 1)

    plt.imshow(image)

    plt.title(train_dataset.classes[labels[i]])

    plt.axis("off")

plt.tight_layout()
plt.show()

model = timm.create_model(
    "vit_base_patch16_224",
    pretrained=True,
    num_classes=len(train_dataset.classes)
)

model = model.to(device)
```

```
print("Number of classes:", len(train_dataset.classes))

images, labels = next(iter(train_loader))

images = images.to(device)
labels = labels.to(device)

outputs = model(images)

print("Input shape:", images.shape)
print("Output shape:", outputs.shape)

import torch.nn as nn
import torch.optim as optim

criterion = nn.CrossEntropyLoss()

optimizer = optim.AdamW(
    model.parameters(),
    lr=3e-5,
    weight_decay=1e-4
)

print("Loss function and optimizer ready.")

scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=10
)

print("Scheduler ready.")

num_epochs = 10

train_losses = []
val_losses = []

train_accuracies = []
val_accuracies = []

best_val_accuracy = 0.0

for epoch in range(num_epochs):

    # TRAINING

    model.train()

    running_loss = 0.0
    correct = 0
    total = 0
```

```
    for images, labels in train_loader:

        images = images.to(device, non_blocking=True)
        labels = labels.to(device, non_blocking=True)

        # Clear previous gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)

        # Calculate loss
        loss = criterion(outputs, labels)

        # Backpropagation
        loss.backward()

        # Update weights
        optimizer.step()

        running_loss += loss.item()

        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)

        correct += (predicted == labels).sum().item()

    train_loss = running_loss / len(train_loader)

    train_accuracy = 100 * correct / total


    # VALIDATION

    model.eval()

    val_running_loss = 0.0
    val_correct = 0
    val_total = 0

    with torch.no_grad():

        for images, labels in val_loader:

            images = images.to(device, non_blocking=True)
            labels = labels.to(device, non_blocking=True)

            outputs = model(images)
```

```
            loss = criterion(outputs, labels)

            val_running_loss += loss.item()

            _, predicted = torch.max(outputs, 1)

            val_total += labels.size(0)

            val_correct += (
                predicted == labels
            ).sum().item()

    val_loss = val_running_loss / len(val_loader)

    val_accuracy = 100 * val_correct / val_total


    # LEARNING RATE

    scheduler.step()


    # SAVE RESULTS

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    train_accuracies.append(train_accuracy)
    val_accuracies.append(val_accuracy)


    # SAVE BEST MODEL

    if val_accuracy > best_val_accuracy:

        best_val_accuracy = val_accuracy

        torch.save(
            model.state_dict(),
            "/content/best_tea_vit.pth"
        )

        print("Best model saved!")


    # PRINT RESULTS

    print(
        f"\nEpoch [{epoch+1}/{num_epochs}]"
    )
```

```
    print(
        f"Train Loss: {train_loss:.4f}"
    )

    print(
        f"Train Accuracy: {train_accuracy:.2f}%"
    )

    print(
        f"Validation Loss: {val_loss:.4f}"
    )

    print(
        f"Validation Accuracy: {val_accuracy:.2f}%"
    )

    print(
        f"Learning Rate: "
        f"{optimizer.param_groups[0]['lr']:.7f}"
    )

print("\nTraining completed!")
print(
    f"Best Validation Accuracy: "
    f"{best_val_accuracy:.2f}%"
)

import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.plot(
    train_accuracies,
    label="Training Accuracy"
)

plt.plot(
    val_accuracies,
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")

plt.title(
    "ViT Training and Validation Accuracy"
)

plt.legend()
plt.grid()
```

```
plt.show()

plt.figure(figsize=(8, 5))

plt.plot(
    train_losses,
    label="Training Loss"
)

plt.plot(
    val_losses,
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.title(
    "ViT Training and Validation Loss"
)

plt.legend()
plt.grid()

plt.show()

model.load_state_dict(
    torch.load(
        "/content/best_tea_vit.pth",
        map_location=device
    )
)

model = model.to(device)

print(
    f"Loaded best model with "
    f"validation accuracy: {best_val_accuracy:.2f}%"
)

model.eval()

correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)
```

```
        outputs = model(images)
        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

test_accuracy = 100 * correct / total

print(f"Test Accuracy: {test_accuracy:.2f}%")

from sklearn.metrics import classification_report
import numpy as np

model.eval()

all_labels = []
all_predictions = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)

        outputs = model(images)
        predictions = torch.argmax(outputs, dim=1)

        all_labels.extend(labels.numpy())
        all_predictions.extend(predictions.cpu().numpy())

# Classification report
print(classification_report(
    all_labels,
    all_predictions,
    target_names=test_dataset.classes,
    digits=4
))

from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(all_labels, all_predictions)

plt.figure(figsize=(9, 7))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=test_dataset.classes,
    yticklabels=test_dataset.classes
```

```
)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix - Tea Leaf Disease ViT")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

from sklearn.metrics import classification_report
import pandas as pd

report = classification_report(
    all_labels,
    all_predictions,
    target_names=test_dataset.classes,
    output_dict=True
)

report_df = pd.DataFrame(report).transpose()

report_df.to_csv(
    "/content/tea_vit_classification_report.csv"
)

print("Classification report saved successfully.")
print(report_df)

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(all_labels, all_predictions)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=test_dataset.classes,
    yticklabels=test_dataset.classes
)

plt.xlabel("Predicted Label", fontsize=12)
plt.ylabel("True Label", fontsize=12)
plt.title("Confusion Matrix - Tea Leaf Disease Classification using
ViT", fontsize=14)
```

```
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)

plt.tight_layout()

plt.savefig(
    "/content/tea_vit_confusion_matrix.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

print("Confusion matrix saved successfully.")

import matplotlib.pyplot as plt
import torch

model.eval()

# Get one batch from test set
images, labels = next(iter(test_loader))

images_gpu = images.to(device)

with torch.no_grad():
    outputs = model(images_gpu)
    predictions = torch.argmax(outputs, dim=1)

# Show first 12 images
plt.figure(figsize=(15, 10))

for i in range(min(12, len(images))):
    image = images[i].permute(1, 2, 0).numpy()

    # Undo normalization
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])

    image = image * std + mean
    image = np.clip(image, 0, 1)

    actual = test_dataset.classes[labels[i].item()]
    predicted = test_dataset.classes[predictions[i].item()]

    plt.subplot(3, 4, i + 1)
    plt.imshow(image)

    title_color = "green" if actual == predicted else "red"

    plt.title(
```

```
        f"Actual: {actual}\nPredicted: {predicted}",
        color=title_color,
        fontsize=10
    )

    plt.axis("off")

plt.suptitle(
    "ViT Tea Leaf Disease Predictions",
    fontsize=16
)

plt.tight_layout()
plt.show()

import numpy as np

cm = confusion_matrix(all_labels, all_predictions)

print("Per-Class Accuracy")
print("=" * 40)

for i, class_name in enumerate(test_dataset.classes):
    class_total = cm[i].sum()
    class_correct = cm[i, i]

    accuracy = 100 * class_correct / class_total

    print(f"{class_name:<20} : {accuracy:.2f}%")

# Create final per-class results table

final_results = report_df.loc[
    test_dataset.classes,
    ["precision", "recall", "f1-score", "support"]
].copy()

final_results.columns = [
    "Precision",
    "Recall",
    "F1-Score",
    "Support"
]

# Convert metrics to percentages
final_results["Precision"] *= 100
final_results["Recall"] *= 100
final_results["F1-Score"] *= 100

final_results = final_results.round(2)
```

```python
print("FINAL ViT MODEL RESULTS")
print("=" * 70)
print(final_results)

print("\nOverall Test Accuracy:", f"{test_accuracy:.2f}%")
print(
    "Macro F1-Score:",
    f"{report_df.loc['macro avg', 'f1-score'] * 100:.2f}%"
)
print(
    "Weighted F1-Score:",
    f"{report_df.loc['weighted avg', 'f1-score'] * 100:.2f}%"
)

# Save results
final_results.to_csv(
    "/content/tea_vit_final_results.csv"
)

print("\nResults saved to:")
print("/content/tea_vit_final_results.csv")

import os
import torch

# Create results folder
results_dir = "/content/tea_vit_results"
os.makedirs(results_dir, exist_ok=True)

# Save best model
torch.save(
    model.state_dict(),
    os.path.join(results_dir, "best_tea_vit.pth")
)

# Copy final results
final_results.to_csv(
    os.path.join(results_dir, "classification_results.csv")
)

# Save confusion matrix
plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=test_dataset.classes,
    yticklabels=test_dataset.classes
```

```
)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix - Tea Leaf Disease Classification using
ViT")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()

plt.savefig(
    os.path.join(results_dir, "confusion_matrix.png"),
    dpi=300,
    bbox_inches="tight"
)

plt.close()

print("All final results saved in:")
print(results_dir)

print("\nFiles:")
for file in os.listdir(results_dir):
    print("-", file)

import matplotlib.pyplot as plt
import os

results_dir = "/content/tea_vit_results"

# Accuracy curve
plt.figure(figsize=(8, 5))

plt.plot(
    train_accuracies,
    label="Training Accuracy"
)

plt.plot(
    val_accuracies,
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title("ViT Training and Validation Accuracy")
plt.legend()
plt.grid(True)

plt.tight_layout()

plt.savefig(
```

```
    os.path.join(results_dir, "accuracy_curve.png"),
    dpi=300,
    bbox_inches="tight"
)

plt.show()


# Loss curve
plt.figure(figsize=(8, 5))

plt.plot(
    train_losses,
    label="Training Loss"
)

plt.plot(
    val_losses,
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("ViT Training and Validation Loss")
plt.legend()
plt.grid(True)

plt.tight_layout()

plt.savefig(
    os.path.join(results_dir, "loss_curve.png"),
    dpi=300,
    bbox_inches="tight"
)

plt.show()

print("Training curves saved successfully.")
```